

PRACTICAL:WEEK-5

Task - 1:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1) {
        perror("Pipe failed");
        return 1;
    }

    // First child executes: ls -l
    pid1 = fork();
    if (pid1 == 0) {
        close(fd[0]); // Close read end
        dup2(fd[1], STDOUT_FILENO); // Redirect standard output to pipe write end
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
        return 0;
    }

    // Second child executes: grep ".c"
    pid2 = fork();
    if (pid2 == 0) {
        close(fd[1]); // Close write end
        dup2(fd[0], STDIN_FILENO); // Redirect standard input to pipe read end
        close(fd[0]);
        execlp("grep", "grep", ".c", NULL);
        return 0;
    }

    // Parent process
    close(fd[0]);
    close(fd[1]);
}
```

```
return 0;
}

// Second child executes: grep ".c"
pid2 = fork();
if (pid2 == 0) {
    close(fd[1]); // Close write end
    dup2(fd[0], STDIN_FILENO); // Redirect standard input to pipe read end
    close(fd[0]);
    execlp("grep", "grep", ".c", NULL);
    return 0;
}

// Parent process
close(fd[0]);
close(fd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;
}
```

OUTPUT

```
pranay@LAPTOP-OP7R42IK: ~$ nano producer_consumer.c
pranay@LAPTOP-OP7R42IK: ~$ gcc producer_consumer.c -o producer_consumer
./producer_consumer
Parent (Producer): Sending data...
Child (Consumer): Receiving data...
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Consumed: 11
Consumed: 12
Consumed: 13
Consumed: 14
Consumed: 15
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20
Consumed: 21
Consumed: 22
Consumed: 23
Consumed: 24
Consumed: 25
Consumed: 26
Consumed: 27
Consumed: 28
Consumed: 29
Consumed: 30
Consumed: 31
Consumed: 32
Consumed: 33
Consumed: 34
Consumed: 35
Consumed: 36

Consumed: 57
Consumed: 58
Consumed: 59
Consumed: 60
Consumed: 61
Consumed: 62
Consumed: 63
Consumed: 64
Consumed: 65
Consumed: 66
Consumed: 67
Consumed: 68
Consumed: 69
Consumed: 70
Consumed: 71
Consumed: 72
Consumed: 73
Consumed: 74
Consumed: 75
Consumed: 76
Consumed: 77
Consumed: 78
Consumed: 79
Consumed: 80
Consumed: 81
Consumed: 82
Consumed: 83
Consumed: 84
Consumed: 85
Consumed: 86
Consumed: 87
Consumed: 88
Consumed: 89
Consumed: 90
Consumed: 91
Consumed: 92
Consumed: 93
Consumed: 94
Consumed: 95
Consumed: 96
Consumed: 97
```

OBSERVATION:

The parent process successfully acted as the producer and generated data, which was sent to the child process through an anonymous pipe. The child process acted as the consumer and successfully received and displayed the data using the read end of the pipe. The communication between the parent and child processes was successfully established using `pipe()` and `fork()`. The time taken for the producer-consumer communication was measured using `clock_gettime()`, allowing the communication efficiency to be observed. The experiment demonstrates that anonymous pipes can be effectively used for communication between related processes

TASK2

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
Communication completed.
Communication time: 0.001910 seconds
pranay@LAPTOP-OP7R42IK:~$ nano pipeline.c
pranay@LAPTOP-OP7R42IK:~$
pranay@LAPTOP-OP7R42IK:~$ nano pipeline.c
pranay@LAPTOP-OP7R42IK:~$ gcc pipeline.c -o pipeline
./pipeline
-rw-r--r-- 1 pranay pranay   80 Jul 25 03:41 Ubuntu.c
-rw-r--r-- 1 pranay pranay  929 Sep 15 06:47 pipeline.c
-rw-r--r-- 1 pranay pranay 1057 Aug  1 09:13 pranay.c
-rwxr-xr-x 1 pranay pranay 16592 Aug  8 03:06 process_states
-rw-r--r-- 1 pranay pranay  2217 Aug  8 03:05 process_states.c
-rwxr-xr-x 1 pranay pranay 16400 Sep 15 06:38 producer_consumer
-rw-r--r-- 1 pranay pranay  1397 Sep 15 06:38 producer_consumer.c
-rw-r--r-- 1 pranay pranay  2816 Aug 11 05:03 task04.c
-rw-r--r-- 1 pranay pranay  1173 Aug 11 06:44 task3.c
-rw----- 1 pranay pranay   472 Aug  9 13:19 task3.c.save
-rw-r--r-- 1 pranay pranay  1159 Aug 10 03:26 task4.c
-rw----- 1 pranay pranay   103 Aug 10 03:24 task4.c.save
-rw-r--r-- 1 pranay pranay  1762 Aug 11 04:59 tasks.c
-rw-r--r-- 1 pranay pranay  5494 Aug 11 05:06 tasks3.c
-rw-r--r-- 1 pranay pranay   756 Aug  4 06:44 week.c
-rw-r--r-- 1 pranay pranay   463 Aug  4 06:41 week3.c
-rw-r--r-- 1 pranay pranay   756 Aug  4 06:45 weekt2.c
pranay@LAPTOP-OP7R42IK:~$ |
```

OBSERVATION:

The program successfully implemented the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The first child process executed `ls -l` and redirected its output to the pipe, while the second child process received the pipe output as its input and executed `grep ".c"`. The required .c files were successfully displayed as the output. The parent process used `waitpid()` to synchronize with both child processes before terminating. Thus, inter-process communication and command piping were successfully demonstrated.

GNU nano 8.7.1

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int fd[2];
    pid_t pid;
    int data;

    struct timespec start, end;

    pipe(fd);

    pid = fork();

    if (pid == 0)
    {
        // Child - Consumer

        close(fd[1]);

        printf("Child (Consumer): Receiving data...\n");

        while (read(fd[0], &data, sizeof(data)) > 0)
        {
            printf("Consumed: %d\n", data);
        }

        close(fd[0]);

        return 0;
    }
    else
    {
        // Parent - Producer

        close(fd[0]);

        clock_gettime(CLOCK_MONOTONIC, &start);

        printf("Parent (Producer): Sending data...\n");

        for (int i = 1; i <= 100; i++)
        {
            write(fd[1], &i, sizeof(i));
        }

        close(fd[1]);

        wait(NULL);
        clock_gettime(CLOCK_MONOTONIC, &end);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        printf("\nCommunication completed.\n");
        printf("Communication time: %f seconds\n", time_taken);

        return 0;
    }
}
```